

Function eval1(ville_cible, df_object, dist, list_objects)

Input: ville_cible (target city), df_object (dataframe of objects), dist (distance), list_objects (list of objects)

Output: Evaluation score

Procedure:

1. **Initialize:** $list_eval \leftarrow$ empty list
2. **For each** obj **in** list_objects:
 - $pd_obj \leftarrow df_object[obj - 1]$
 - $weight \leftarrow pd_obj["Weight"]$
 - $profit \leftarrow pd_obj["Profit"]$
 - $list_eval.append\left(\frac{profit}{weight}\right)$
3. $max_eval \leftarrow \max(list_eval)$
4. **If** dist == 0, **return** max_eval
5. **Otherwise, return** $\frac{max_eval}{dist}$

Function eval2(ville_cible, df_object)

Input: ville_cible (target city), df_object (dataframe of objects)

Output: Weight of the object in the target city

Procedure:

1. $obj1 \leftarrow df_object["City_Index" == ville_cible]$
2. $weight \leftarrow obj1["Weight"].values[0]$
3. **return** weight

Algorithm 1 Algorithme Glouton

Require: df_ville , df_object , $capacite$ **Ensure:** Liste des villes visitées π , objets pris obj_pris , poids total $poids_tot$, dictionnaire des objets pris par ville $dict_ville_objet_pris$

```
1:  $poids\_tot \leftarrow 0$ 
2:  $nb\_objet \leftarrow$  longueur des objets dans  $df\_object$ 
3:  $obj\_pris \leftarrow$  tableau de zéros de taille  $nb\_objet$ 
4:  $nb\_ville \leftarrow$  longueur des villes dans  $df\_ville$ 
5:  $critere\_limite \leftarrow capacite / (nb\_objet * 100)$ 
6:  $villes\_dispo \leftarrow$  liste des indices des villes dans  $df\_ville$ 
7:  $ville\_actuelle \leftarrow 1$ 
8:  $deposed \leftarrow True$ 
9: Retirer  $ville\_actuelle$  de  $villes\_dispo$ 
10: Initialiser les dictionnaires  $dict\_ville\_objet$  et  $dict\_ville\_objet\_pris$ 
11: while  $capacite > poids\_tot$  and  $villes\_dispo \neq \emptyset$  and  $deposed$  do
12:   Calculer  $distance\_table \leftarrow$  distances de toutes les villes depuis
    $ville\_actuelle$ 
13:    $ville\_cible\_best \leftarrow$  ville dans  $villes\_dispo$  maximisant les evaluations en
   ordre lexicographique d'eval 1 et 2 (voir les fonctions eval1 et eval2)
14:    $deposed \leftarrow False$ 
15:   Trier les objets disponibles dans  $ville\_cible\_best$  par rapport au rapport
    $\frac{Profit}{Poids}$ 
16:   for chaque objet  $obj$  dans la liste triée do
17:     if  $poids\_obj \leq capacite - poids\_tot$  then
18:        $deposed \leftarrow True$ 
19:       if  $critere\_limite > eval\_objet$  then
20:         Ajouter  $obj$  à  $dict\_ville\_objet\_pris[ville\_cible\_best]$ 
21:          $poids\_tot \leftarrow poids\_tot + poids\_obj$ 
22:         Mettre à jour  $obj\_pris$ 
23:       end if
24:     end if
25:   end for
26:   Mettre à jour  $\pi$  et  $ville\_actuelle$ 
27: end while
28: while  $villes\_dispo \neq \emptyset$  do
29:   Calculer  $distance\_table \leftarrow$  distances de toutes les villes depuis
    $ville\_actuelle$ 
30:    $ville\_cible\_best \leftarrow$  ville dans  $villes\_dispo$  minimisant la distance
31:   Mettre à jour  $\pi$  et  $ville\_actuelle$ 
32: end while
33:  $\pi$  reversed
34: Retourner  $\pi$ ,  $obj\_pris$ ,  $poids\_tot$ ,  $dict\_ville\_objet\_pris$ 
```
